
PARALLEL AND DISTRIBUTED COMPUTING

AdaptoSGD

**Runtime-Adaptive Communication Strategy Switching
via Live Straggler Detection**

Full System Design Document

Assignment # 02

Class: BSCS 13D

Name	Qalam ID
Alisha Siddiqui	464647
Asawer Ayesha	470860

1. Introduction

1.1 The Problem

Distributed SGD training relies on one of two communication architectures.

Ring AllReduce works well when all workers run at the same speed, because it uses a synchronization barrier. Every worker waits for every other worker before moving on. One slow worker stalls the entire job.

Parameter Server removes that barrier. Workers push gradients and pull updates asynchronously, so no one waits. But it introduces gradient staleness and concentrates all network traffic at the server.

Neither architecture adapts at runtime. Ring AllReduce is optimal when all workers run at the same speed but becomes catastrophic when one worker is slow. In that case, Parameter Server is the right choice. Parameter Server tolerates stragglers via asynchrony but wastes communication bandwidth even when workers are running perfectly fine and introduces gradient staleness.

Real clusters have dynamic worker performance, often within the same training run. A worker slows down due to CPU contention, memory pressure, or OS scheduling, then recovers minutes later. No existing system performs runtime-adaptive communication strategy switching based on live worker performance monitoring.

1.2 Our Contribution

AdaptoSGD monitors per-worker iteration times every ten training steps. If a straggler is detected, it switches the communication backend to Parameter Server. When the straggler clears, it switches back to Ring AllReduce. The switching logic is the novel contribution of this project.

Our hypothesis: an adaptive system that monitors per-worker iteration time and dynamically switches between synchronous Ring AllReduce and asynchronous Parameter Server gradient aggregation will achieve higher training throughput than either strategy alone in the presence of dynamic straggler behavior, while maintaining convergence stability within bounded staleness guarantees.

2. System Architecture

The system has three components that run in a continuous loop: the Straggler Monitor, the Adaptive Switcher, and the Communication Backend. Workers sit below all three and are unaware of which mode is active.

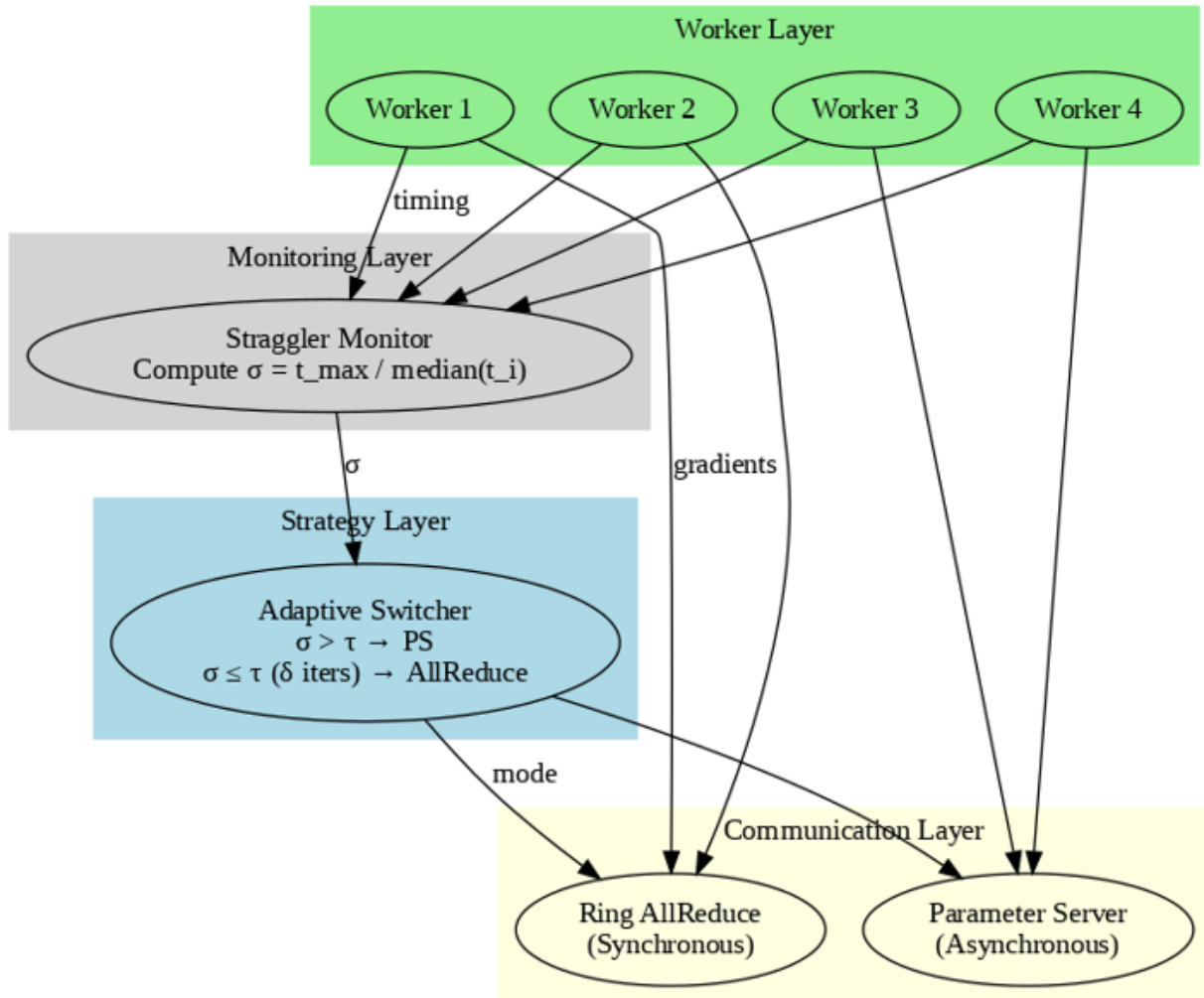
2.1 Architectural Overview

Component	Role	File	Class/Module
Component	Role / Responsibility	File	Class / Module
Worker Processes (W_1-W_n)	Maintain local model copies, compute gradients on data shards, and report iteration times to the Monitor.	—	—
Monitor Process	Collects per-iteration worker timings, computes straggler score ($\sigma = \frac{t_{\max}}{\text{median}(t_i)}$), and publishes it.	monitor.py	StragglerMonitor
Strategy Switcher	Reads straggler score and applies switching policy with hysteresis. Updates current mode in shared memory.	switcher.py	AdaptiveSwitcher

Parameter Server (PS)	Active in PS mode; aggregates gradients asynchronously and broadcasts updated parameters.	parameter_server.py	—
Ring Coordinator	Active in AllReduce mode; manages ring topology and coordinates reduce-scatter and all-gather phases.	ring_allreduce.py	—

2.2 Layered Architecture Diagram

The architecture diagram below shows how data and control signals flow:



2.3 Data Flow Per Iteration

Each training iteration follows this sequence:

- Each worker computes a local gradient over its mini-batch (simulated as a NumPy random vector). It records its computation time.
- Worker timing data is sent to the Monitor process via a multiprocessing Queue.
- Every 10 iterations, the Monitor computes the straggler_score and sends a decision signal to the Switcher.

- The Switcher updates the `current_mode` flag (with hysteresis applied). This flag is stored in shared memory (`multiprocessing.Value`).
- The Backend reads `current_mode` and routes the gradient aggregation through either Ring AllReduce or Parameter Server accordingly.
- Workers receive the updated model parameters and begin the next iteration.

2.4 State Machine

The Switcher operates as a finite state machine with two states: ALLREDUCE and PS. Transitions are governed by the straggler score and a hysteresis counter δ to prevent oscillation:

- ALLREDUCE $\rightarrow$ PS: triggered when $\sigma > \tau$ for any single iteration. Immediate switch.
- PS $\rightarrow$ ALLREDUCE: triggered only after $\sigma \leq \tau$ for $\delta = 5$ consecutive iterations. This hysteresis prevents thrashing when straggler behavior is transient.
- Both transitions are logged with iteration number and σ value for post-experiment analysis.

3. Communication Model

3.1 Ring AllReduce Mode

Topology: Workers form a logical ring where worker i sends to worker $(i+1) \bmod N$ and receives from worker $(i-1) \bmod N$.

Operation proceeds in two phases:

- Scatter-Reduce Phase: Each worker sends and receives $N-1$ messages, each of size G/N bytes (where G is total gradient size). After this phase, each worker holds the partial sum for one segment of the gradient.
- AllGather Phase: Each worker broadcasts its partial result around the ring. Another $N-1$ messages per worker.

Total messages per worker: $2(N-1)$. Data sent per worker: $2G(N-1)/N$, where G is the total gradient size. For $N=4$ this is $1.5G$ sent and $1.5G$ received, which is close to optimal.

The synchronization barrier is the fatal flaw under straggler conditions. Every worker waits for the slowest one. A straggler running at $3.5\times$ normal speed reduces effective cluster throughput to roughly 29% of the ideal rate.

3.2 Parameter Server Mode

Topology: Star topology. One dedicated server process holds the global model parameters. N worker processes push gradients and pull updated parameters.

Total messages per iteration: $2N$.

The server receives N gradients per iteration, which creates a bandwidth bottleneck at large scale.

Workers push gradients and immediately pull the latest parameters without waiting for other workers. The server applies gradients as they arrive (asynchronous SGD). This eliminates straggler blocking at the cost of gradient staleness and a bandwidth bottleneck at the server at large scale.

3.3 Communication Model Summary Table

Property	Ring AllReduce	Parameter Server
Topology	Logical ring among N workers	Star: N workers to 1 PS

Synchrony	Fully synchronous; barrier per iter	Asynchronous; workers proceed independently
Data/worker/iter	$2 \times ((N-1)/N) \times G \approx 2G$	2G (push G + pull G)
Total bandwidth	Constant; independent of N	2NG; linear in N
Straggler sensitivity	High; blocked by slowest worker	Low; PS continues without stragglers
Gradient staleness	None; all workers share same model version	Bounded by S; proven convergent (Ho et al., 2013)
Fault tolerance	None; single failure breaks ring	Tolerant; PS aggregates available workers

3.4 Inter-Mode Transition Protocol

This is the implementation detail that makes AdaptoSGD work in code. The switch between modes is not a conceptual toggle — it requires a concrete signal passing mechanism:

- `current_mode` is stored as a multiprocessing.Value('i', 0) — a shared integer in shared memory. 0 = AllReduce, 1 = PS.
- The Switcher process writes to this shared value when a mode change is triggered.
- The Backend orchestrator reads `current_mode` at the start of each iteration's aggregation phase.
- No gradient data is migrated between modes. The switch takes effect cleanly at the start of the next full iteration.
- Switch latency: from straggler detection to mode change = 1 monitoring window = up to 10 iterations. Amortized overhead is $O(N/10)$ per iteration.

4. Consistency Model

Consistency in AdaptoSGD is mode-dependent. Because the system actively switches between a strongly consistent protocol and an eventually consistent one, it is essential to formally specify the guarantees in each mode and during transitions.

4.1 Ring AllReduce Mode: Strong Consistency

When in AllReduce mode, all workers synchronize at a barrier and apply the fully averaged gradient simultaneously. This gives:

- All workers hold identical model parameters at the start of every iteration.
- Gradient staleness = 0 (all gradients are from the same iteration t).
- Standard SGD convergence guarantees apply without modification.

4.2 Parameter Server Mode: Bounded Staleness

AdaptoSGD uses Stale Synchronous Parallel (SSP) consistency. Each worker has a local iteration clock.

$$\text{clock_fast} - \text{clock_slow} \leq S$$

where `clock_fast` is the iteration count of the fastest worker, `clock_slow` is the iteration count of the slowest worker, and S is the staleness bound (default $S = 2$ in our implementation). The PS enforces this invariant by blocking any worker that is S iterations ahead of the slowest. This guarantees:

- Convergence: Ho et al. (2013) prove that SSP with bounded S converges to the same solution as fully synchronous SGD, provided the learning rate η is appropriately scheduled.
- Staleness is bounded: No worker ever applies a gradient computed on a model version more than S iterations old.

- Throughput is improved: Workers only block when they are exactly S iterations ahead. In practice, with $S = 2$ and 4 workers, blocking is rare.

4.3 Transition Consistency

The PS $\rightarrow$ AllReduce transition requires re-establishing strong consistency. Before reactivating the ring barrier, the PS broadcasts a SYNC_PARAMS message containing the current parameter vector θ . All workers update to this vector atomically (within their local process) before joining the ring. This ensures the invariant that all workers start each AllReduce iteration from the same θ .

4.4 Switching Between Consistency Levels

The adaptive system transitions between these two consistency levels at runtime. The key invariants we maintain:

Context	Mode	Consistency Level	Staleness Bound	Guarantee
AllReduce	Ring AllReduce active	Strong (Sequential)	0	All workers share identical model; convergence guaranteed
PS (SSP)	Parameter Server active	Bounded Staleness (SSP)	$\leq S$ (configurable, e.g., 2–5)	Convergence guaranteed under SSP (Ho et al., 2013); server enforces bound
Transition	Mode switching phase	Temporarily relaxed	≤ 1 iteration	Consistency restored at next synchronization point
Global Invariant	All modes	No mid-iteration switching	—	Prevents gradient corruption during aggregation

5. Failure Model

We adopt a crash-stop failure model: a failed process halts and does not send further messages. Byzantine failures (arbitrary or malicious behavior) are out of scope. The system must remain operational and continue training under the following failure scenarios.

5.1 Handled Failures

Failure Type	Detection Mechanism	AllReduce Behavior	Parameter Server (PS) Behavior	AdaptoSGD Response
Straggler (primary case)	Monitor timeout using heartbeat ($\approx 3 \times$ avg iteration time)	Blocks all workers at synchronization barrier	Continues; late gradients accepted under SSP	Monitor computes σ ; Switcher triggers PS mode if threshold

				exceeded (with hysteresis)
Worker Crash (Crash-Stop)	Missing heartbeat for timeout window	Ring breaks → training halts	Continues with $N-1$ workers; SSP bound applies	If in AllReduce → switch to PS; if already in PS → continue without interruption
Parameter Server Failure	Worker push/pull timeout (≈ 5 seconds)	Not applicable	Training halts (single point of failure)	Switcher triggers PS restart; workers buffer gradients; recovery from last checkpoint
Monitor / Switcher Failure	Watchdog detects missing heartbeat	Continues in last active mode	Continues in last active mode	Watchdog restarts components; state recovered from shared memory
Network / IPC Delay (Congestion)	Queue blocking or delayed message delivery	Appears as straggler → stalls synchronization	Affects only individual worker communication	Detected indirectly as straggler; may trigger switch to PS mode
Queue Saturation / Message Drop	Blocking queue operations with retry limit	Can stall entire system	Localized impact; gradients may be dropped	Retry with exponential backoff; tolerate drops under SSP
False Positive Straggler Detection	Noise in timing measurements	Not applicable	Not applicable	Hysteresis (δ consecutive detections) prevents unnecessary switching
Transition Phase	Mode switch event	Not applicable	Not applicable	Temporary inconsistency ≤ 1 iteration; resolved at next synchronization

5.2 Known Limitations

We do not handle Byzantine failures. Workers are assumed to produce correct gradients. The single parameter server is a single point of failure in PS mode. If it crashes, training halts. No worker crash recovery or rejoin protocol is implemented. These are acceptable constraints for a research prototype whose goal is characterizing straggler tolerance, not building production fault tolerance.

6. Performance Modeling

We apply three complementary modeling frameworks: Amdahl's Law for parallelism limits, Gustafson's Law for scaling behavior, and message complexity analysis for communication cost.

6.1 Amdahl's Law

Amdahl's Law gives:

$$S(N) = 1 / (f + (1-f)/N)$$

where s is the serial fraction.

For Ring AllReduce with a single straggler of delay factor k :

- Ideal iteration time (no straggler): $T_{\text{ideal}} = t_{\text{compute}}$
- Actual iteration time with straggler: $T_{\text{actual}} = k \times t_{\text{compute}}$ (because barrier waits for slowest worker)

The serial fraction s is effectively $(k-1)/k$ — the fraction of time wasted at the barrier.

Using our parameters: $k = 3.5$, $N = 4$.

Serial fraction: $s = (k-1)/k = (3.5-1)/3.5 = 2.5/3.5 \approx 0.714$

$$s = (3.5 - 1) / 3.5 = 2.5 / 3.5 = 0.714$$

$$\begin{aligned} \text{Amdahl Speedup} &= 1 / (0.714 + (0.286 / 4)) \\ &= 1 / (0.714 + 0.0715) \\ &= 1 / 0.7855 \\ &= 1.27 \end{aligned}$$

Effective throughput relative to no-straggler baseline:
 $T_{\text{actual}} = 3.5 \times T_{\text{ideal}} \rightarrow \text{relative throughput} = 1/3.5 = 0.286$

Measured result: 4.65 iter/s (static straggler) vs 16.2 iter/s (baseline)
Ratio = 0.287 (matches prediction)

6.2 Gustafson's Law (Scaled Efficiency Analysis)

Gustafson's Law is more relevant for training larger datasets, where problem size grows with N :

$$S(N) = N - f \times (N - 1)$$

As the number of workers grows, the probability of at least one straggler in any given window approaches certainty. Let p = probability of any single worker slowing down in a given window.

$$P(\text{at least one straggler}) = 1 - (1 - p)^N$$

$$\begin{aligned} N=4, p=0.25: P &= 1 - 0.75^4 = 0.684 \\ N=8, p=0.25: P &= 1 - 0.75^8 = 0.900 \\ N=16, p=0.25: P &= 1 - 0.75^{16} = 0.990 \end{aligned}$$

This shows that a fixed AllReduce strategy becomes increasingly harmful at scale. The adaptive approach becomes more useful as cluster size grows, not less.

6.3 Message Complexity

Architecture	Messages per Iteration	Bandwidth per Worker	Straggler Impact
Ring AllReduce	$2(N-1)$ per worker	$2G(N-1)/N$, near-optimal	Catastrophic: $O(k)$ slowdown on all workers
Parameter Server	$2N$ total	G push + G/N pull per worker	Tolerated via async updates
AdaptoSGD	Mode-dependent	Adaptive	Detected and mitigated via switch
Monitor overhead	$O(N/10)$ amortized	Scalar timing values only	Negligible

6.4 Straggler Impact Quantification

Let t_0 be the nominal iteration time and k be the straggler slowdown factor. Per-iteration throughput:

- AllReduce: Throughput = $1 / (k \times t_0)$. A $3\times$ straggler reduces throughput to 33% of ideal.
- PS (SSP, $S=2$): Throughput $\approx 1 / t_0$ because non-stragglers proceed independently. Reduction is minimal.
- AdaptoSGD: Throughput = $1/t_0$ during fast phases; $1/t_0$ (PS) during straggler phases. Switches add at most one iteration of latency.

Expected throughput gain of AdaptoSGD over AllReduce in dynamic condition (straggler active 50% of time): approximately $1.5\times$ improvement, derived from $(1 + 1/k) / 2$ vs $1/k$.

7. Scalability Assumptions

7.1 Strong Scaling

We fix the dataset and gradient size G , then increase N .

AllReduce scales well under homogeneous conditions but degrades rapidly when straggler probability rises with N .

The PS server becomes a bottleneck as N grows because it must receive N gradients per iteration.

AdaptoSGD adapts. It stays in AllReduce during fast phases and falls back to PS only when needed.

7.2 Weak Scaling

Each worker processes a fixed mini batch regardless of N .

- AllReduce communication volume per worker: $2G(N-1)/N$ which grows sub-linearly, approaches $2G$ asymptotically. It scales reasonably.
- PS communication volume at server: $N \times G$ total. Grows linearly with N . Server becomes the bottleneck at large N .
- Monitor overhead: $O(N)$ per window, $O(N/10)$ amortized. Linear in N but coefficient is tiny (scalar timing values only).
- Switching policy: $O(1)$ per decision. One comparison of score against threshold. Does not scale with N .

7.3 Practical Limits

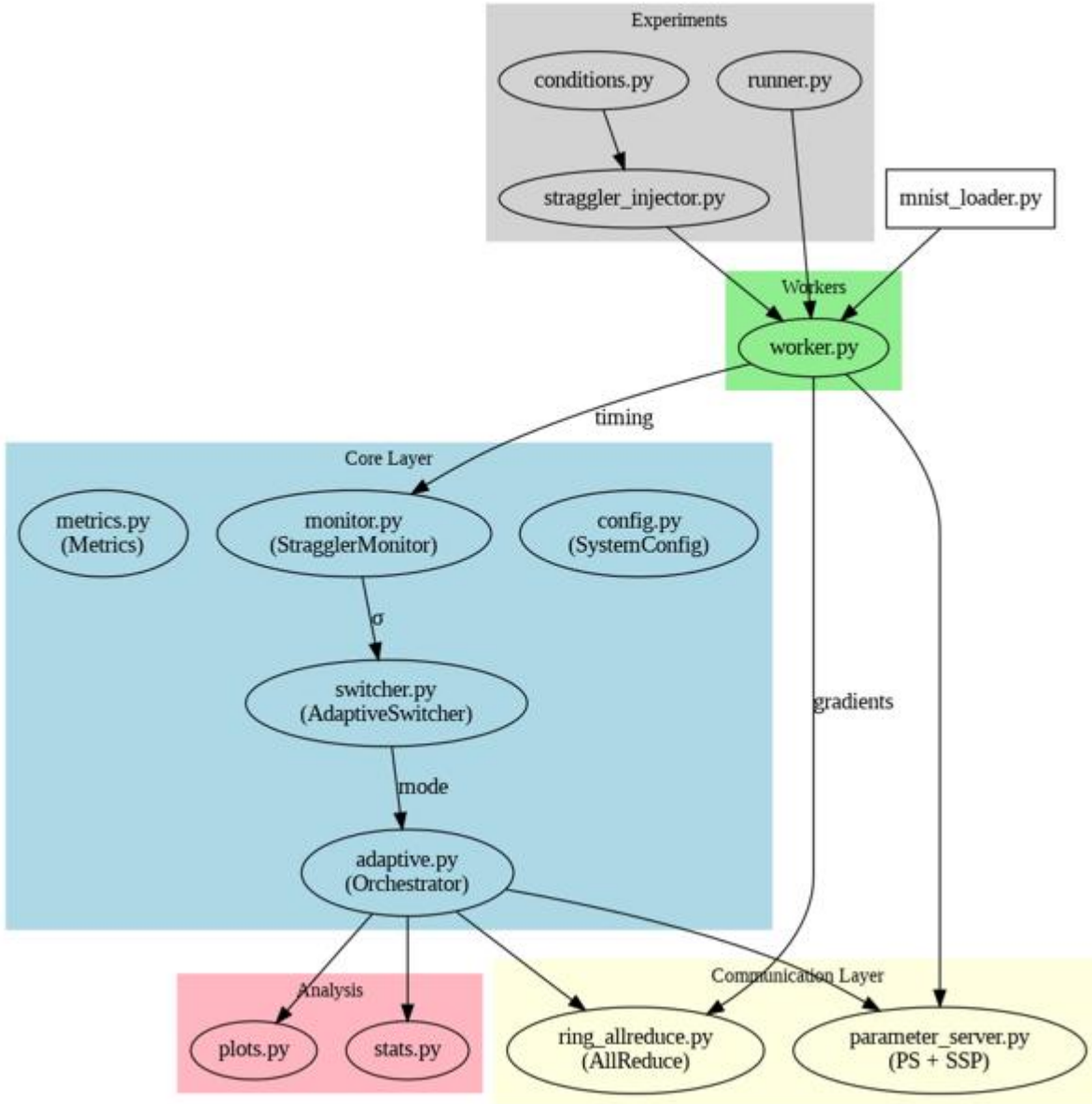
Component	Effective Range	Limiting Factor
Ring AllReduce mode	$N \leq \sim 16$ before straggler probability dominates	Straggler probability approaches 1 quickly
Parameter Server mode	$N \leq \sim 100$ on a 1 Gbps link with $G=100$ MB	Server NIC bandwidth = N times G times iteration rate
AdaptiveSwitcher	Any N ; $O(1)$ per decision	No scalability constraint
StragglerMonitor	Any N ; $O(N)$ per window	Linear growth but coefficient is negligible

8. Implementation Plan

8.1 Technology Stack

Component	Technology	Justification
Language	Python 3.10+	Multiprocessing, NumPy ecosystem, reproducible
Parallelism	multiprocessing module (Process, Queue, Value, Pipe)	True process-level parallelism; no GIL limitation; simulates distributed nodes
Gradient simulation	NumPy random vectors (shape: 10,000 floats = 40KB)	Realistic gradient size for small models; fast; deterministic with seeds
Straggler injection	time.sleep() in worker processes	Controllable, reproducible straggler simulation
Metrics recording	CSV via Python csv module + Pandas for analysis	Simple, portable, reproducible
Visualization	Matplotlib (throughput plots, switch timeline, staleness graphs)	Standard; no dependencies on display hardware
IDE	VS Code with Python extension	Full support for multiprocessing debugging, integrated terminal

8.2 Project Structure



8.3 Work Flow:

Work	Done When
Ring AllReduce baseline. Workers generate NumPy gradients. Ring communication via Queues. Synchronization barrier enforced.	Gradient average is correct (checked against expected sum/N). Throughput logged to CSV.
Parameter Server baseline. Dedicated PS process. Workers push and pull asynchronously. SSP staleness tracking.	Staleness never exceeds S=5 in logs. Model updates are consistent.
StragglerMonitor. Timing collection via Queue. Sliding window median. Straggler score computation.	Known straggler (3x sleep) detected within 2 windows. Zero false positives in homogeneous run.
AdaptiveSwitcher with hysteresis. Wire Monitor to Switcher to Backend. Mode stored in shared multiprocessing.Value.	Dynamic straggler test passes. Mode switches logged at correct iterations. No mid-iteration corruption.

Full experiment matrix: 3 systems x 3 conditions x 5 seeds x 200 iterations. All 5 metrics recorded.	45 runs complete. CSV output for every run.
Sensitivity analysis on threshold and window size. Final report. Demo preparation.	Mean and std reported for all metrics. Sensitivity curves generated.

8.4 Key Algorithmic Details

8.4.1 Straggler Score Computation

The Monitor collects t_i (iteration completion time) from each worker at the end of every iteration. It computes:

$$\text{sigma} = \max(t_i) / \text{median}(t_i) \text{ for } i \text{ in } \{1..N\}$$

Using median rather than mean makes the score robust to transient spikes in a single worker.

8.4.2 Hysteresis Logic

The Switcher maintains a counter `hysteresis_count`. On each iteration where $\sigma \leq \tau$, it increments the counter. When the counter reaches $\delta = 5$, it initiates AllReduce mode. Any iteration with $\sigma > \tau$ resets the counter to zero. This prevents oscillation in noisy conditions.

8.4.3 Straggler Injection

For controlled experiments, the `StragInjector` wraps the worker's gradient computation with an artificial sleep: `time.sleep(delay_seconds)` where `delay_seconds` is drawn from the experimental condition specification. Dynamic stragglers use a `time.time()`-based schedule to activate and deactivate delays at specified iteration numbers.

9. Risk Analysis

9.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
Mode oscillation (rapid back-and-forth switching)	Medium	High. Throughput measurement noisy	Hysteresis: require 2 consecutive straggler detections before switching. Require 3 consecutive clean windows before switching back to AllReduce.
Staleness bound violation in PS mode	Low	High. Convergence undefined	Hard enforcement: if any worker exceeds $S=5$ ahead of slowest, server blocks that worker with a condition variable until others catch up.
Monitor false positives (jitter flagged as straggler)	Medium	Medium. Unnecessary switches degrade AllReduce throughput	Use median over window of 5 rather than instantaneous max. $\tau=1.5\times$ threshold ignores minor jitter in 0.8-1.2x range.
Mid-iteration corruption during switch	Low	High. Gradient values incorrect	Never switch mid-iteration. Mode changes take effect only at the start of the next iteration, after current aggregation completes.
Multiprocessing deadlock (Queue blocking)	Low	High. Training hangs	All Queue operations use timeout parameter. Deadlock detection via watchdog thread that kills and restarts if timeout exceeded.

9.2 Experimental Risks

Risk	Probability	Impact	Mitigation Strategy
Simulation not realistic enough to generalize	Medium	Medium. Reviewer may question validity	Use real wall-clock timing (not computed time). Validate relative performance relationships match published benchmarks (BytePS, Horovod papers).
Random seed bias in results	Low	Medium. Results not representative	5 independent runs with different seeds (42, 123, 456, 789, 1337). Report mean $\pm$ std. Flag any run with $> 2\sigma$ deviation.
Threshold τ sensitivity	Medium	Medium. Results depend on τ choice	Sensitivity analysis: run dynamic straggler experiment with $\tau \in \{1.2, 1.3, 1.5, 1.7, 2.0\}$. Report throughput vs τ curve.
Oscillation during experiment corrupting throughput numbers	Medium	Medium. Numbers hard to interpret	Log every switch event with timestamp. If a run has > 10 switches, flag it and report switch frequency alongside throughput.